

Lezione 4 - Design Pattern

Design Pattern — Ripasso e Nuovi Pattern

Abbiamo già visto cosa sono i [Design Pattern](#), le loro categorie e alcuni esempi concreti — in particolare il [Singleton](#), l'[Adapter](#) e lo [Strategy](#).

Ora ampliamo il repertorio con quattro nuovi pattern: il **Singleton** (che riprendiamo), il **Factory Method**, il **Proxy** e l'**Observer** e il **Composite**.

Vale la pena ricordare una cosa fondamentale prima di procedere:

- **i Design Pattern sono uno strumento di progettazione, non una soluzione da applicare meccanicamente.**

Un buon sviluppatore dovrebbe conoscerne almeno i principali — quelli catalogati dal [GoF \(Gang of Four\)](#) — e soprattutto sapere **quando** usarli.

Il modo migliore per ricordare e riconoscere un pattern è legarlo al tipo di scenario che risolve.

Nota:

il pattern **Proxy** non è del tutto nuovo — lo abbiamo già incontrato indirettamente con la [Dependency Injection](#) di Spring. Quando Spring usa le annotation come `@Autowired` per iniettare le dipendenze, crea internamente un oggetto proxy che "avvolge" il bean da iniettare. Ora vedremo il pattern in modo esplicito e formale.

Le Famiglie dei Design Pattern

I Design Pattern si dividono in famiglie in base al **tipo di problema** che risolvono. Abbiamo già visto:

- **[Pattern Strutturali](#):**
 - **riguardano la composizione delle classi e degli oggetti.**
 - Abbiamo visto l'[Adapter](#).
 - E aggiungeremo il **Composite** e il **Proxy**
- **[Pattern Comportamentali](#):**
 - riguardano la comunicazione e il comportamento tra gli oggetti.

- Abbiamo visto lo **Strategy**.
Oggi aggiungiamo:
- **Pattern Creazionali**:
 - riguardano il momento della creazione degli oggetti, fornendo meccanismi per crearli in modo flessibile e disaccoppiato.
 - In questa famiglia troviamo il **Singleton** e il **Factory Method**.

Pattern Creational

1. Factory Method

Il problema che risolve: A volte si ha la necessità di creare un oggetto di cui si conosce staticamente l'interfaccia o la superclasse, ma la classe concreta effettiva viene decisa dinamicamente a runtime.

È una situazione tipica dei framework — e non a caso è un pattern che abbiamo già incontrato indirettamente con Spring.

La soluzione: L'idea è semplice ma potente: separare chi crea gli oggetti da chi li usa.

È come con i vestiti — non li produciamo noi, li compriamo e li usiamo. Qualcuno si occupa della fabbricazione, qualcun altro del consumo. Applicato al codice, questo si traduce in:

- Definire l'interfaccia degli oggetti da creare — chi usa l'oggetto conosce solo l'interfaccia, non la classe concreta
- Incapsulare la logica di creazione in una classe Factory dedicata, che si occupa di istanziare gli oggetti specifici che implementano quell'interfaccia
- Chi usa gli oggetti non deve conoscere il tipo esatto — sa solo che l'oggetto rispetta il contratto dell'interfaccia

Il collegamento con Spring:

Questo pattern sta alla base di tutti i framework che usano l'**IoC** — e Spring ne è l'esempio perfetto.

Le annotation come `@Component`, `@Service` e `@Repository` fungono da **interfaccia** verso Spring:

- dicono al framework quale tipo di oggetto creare, mentre Spring — tramite il suo Container — si occupa di istanziare la classe concreta

nel momento giusto.

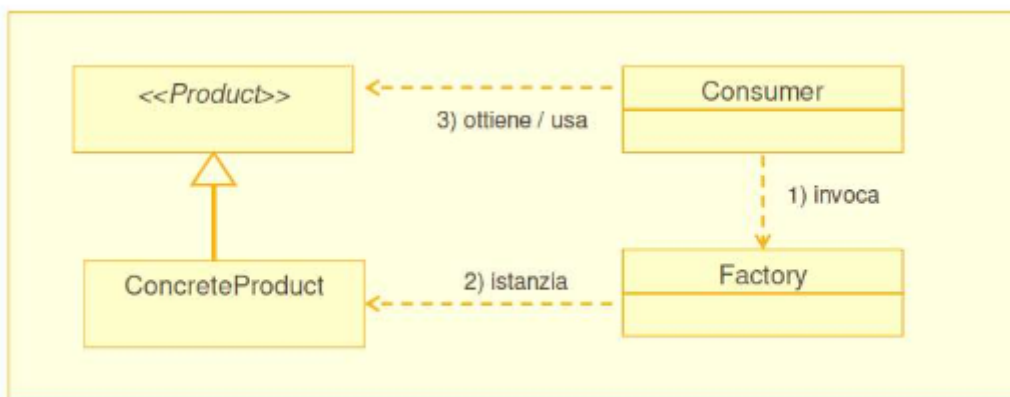
- Chi usa l'oggetto (ad esempio il Controller che riceve il Service tramite `@Autowired`) non sa e non gli importa come è stato creato — sa solo che rispetta il contratto dell'interfaccia.

In sintesi:

il Factory Method è il pattern che permette di disaccoppiare la creazione degli oggetti dal loro utilizzo — esattamente il principio che abbiamo visto con la Dependency Injection di Spring.

Diagramma Facotry Method

Il diagramma mostra chiaramente la separazione dei ruoli che abbiamo descritto:



1. Il Consumer invoca la Factory chiedendole di creare un oggetto
2. La Factory istanzia un oggetto di `ConcreteProduct` — la classe concreta che implementa l'interfaccia `Product`
3. Il Consumer ottiene e usa l'oggetto, ma lo vede solo attraverso il tipo astratto `Product` — non sa e non gli importa che sia un `ConcreteProduct`

Il punto chiave è:

- il Consumer è completamente disaccoppiato dal `ConcreteProduct` — conosce solo l'interfaccia `Product`.
- È la Factory a fare da intermediario tra chi crea e chi usa

✓ Vantaggi del Factory Method

1. Polimorfismo trasparente:

- grazie al polimorfismo, ogni oggetto concreto opera secondo la sua "natura" senza che questa debba essere nota a priori a chi lo usa.
- Il Consumer chiama i metodi dell'interfaccia `Product` e il comportamento corretto viene eseguito automaticamente in base al tipo concreto dell'oggetto.

2. Facilmente estensibile:

- aggiungere un nuovo tipo di prodotto significa semplicemente creare un nuovo `ConcreteProduct` che implementa `Product` e aggiornare la Factory.
- Il Consumer non va toccato — non sa nulla della nuova classe concreta.

3. Incapsulamento delle politiche di creazione:

- la Factory può implementare logiche di creazione più sofisticate, ad esempio:
- **Un solo oggetto** — è quello che fa Spring con i bean annotati con `@Component`:
 - crea una sola istanza della classe Controller e la riusa per tutte le richieste
- **Un pool di oggetti** — è quello che succede tipicamente con le connessioni al database:
 - invece di creare e distruggere una connessione per ogni operazione, una classe gestisce un pool di connessioni già pronte che vengono "prestate" al `DAO` quando ne ha bisogno e restituite al pool una volta terminate.

Esempio pratico — Factory Method in Java

Supponiamo di dover sviluppare un sistema di creazione di prodotti per un e-commerce.

Vediamo come le classi si mappano sui ruoli del diagramma che abbiamo visto:

1. L'interfaccia `Prodotto` :



Java

```
1 package designPattern.factoryMethod;
2
```

```
3 public interface Prodotto {
4     public double getCosto();
5     public double getPrezzo();
6 }
```

1. La classe TShirt



Java

```
1 package designPattern.factoryMethod;
2
3 public class TShirt implements Prodotto {
4     private String taglia;
5     private double costo, prezzo;
6     public TShirt() {}
7
8     public TShirt(double costo, double prezzo, String taglia) {
9         this.costo = costo;
10
11         this.prezzo = prezzo;
12
13         this.taglia = taglia;
14
15     }
16     @Override
17     public double getCosto() {
18         return costo;
19     }
20
21     @Override
22     public double getPrezzo() {
23         return prezzo;
24     }
25
26     @Override
27     public String toString() {
28
29         return "TShirt [taglia=" + taglia + ", costo=" + costo +
30         ", prezzo=" + prezzo + "];"
31     }
```

2. La classe Zaino



```
1  package designPattern.factoryMethod;
2
3  public class Zaino implements Prodotto {
4      private double costo, prezzo;
5      private String marca;
6      public Zaino() {}
7      public Zaino(double costo, double prezzo, String marca) {
8          this.costo = costo;
9          this.prezzo = prezzo;
10         this.marca = marca;
11     }
12     public String getMarca() {
13         return marca;
14     }
15     public void setMarca(String marca) {
16         this.marca = marca;
17     }
18     public void setCosto(double costo) {
19         this.costo = costo;
20     }
21
22     public void setPrezzo(double prezzo) {
23         this.prezzo = prezzo;
24     }
25
26     @Override
27
28     public double getCosto() {
29         return costo;
30     }
31
32     @Override
33     public double getPrezzo() {
34         return prezzo;
35     }
36
37     @Override
38     public String toString() {
39         return "Zaino [costo=" + costo + ", prezzo=" + prezzo + ",
40         marca=" + marca + "];"
41     }
42 }
```

3. Classe Libro



Java

```
1  package designPattern.factoryMethod;
2
3  public class Libro implements Prodotto {
4      private String titolo;
5      private double costo, prezzo;
6      public Libro() {}
7      public Libro(String titolo, double costo, double prezzo) {
8          this.titolo = titolo;
9          this.costo = costo;
10         this.prezzo = prezzo;
11     }
12     @Override
13     public double getCosto() {
14         return costo;
15     }
16     @Override
17     public double getPrezzo() {
18         return prezzo;
19     }
20
21     @Override
22     public String toString() {
23         return "Libro [titolo=" + titolo + ", costo=" + costo + ",
24         prezzo=" + prezzo + "];"
25     }
26 }
```

Quindi l'interfaccia `Prodotto` è il contratto che tutti i prodotti devono rispettare. Il Consumer conosce solo questa interfaccia — non sa nulla delle classi concrete.

`Zaino`, `TShirt`, `Libro` — i **prodotti concreti** (`ConcreteProduct`) Sono le tre implementazioni concrete dell'interfaccia `Prodotto` .

Ognuna ha i propri campi specifici — `Zaino` ha `marca`, `TShirt` ha `taglia`, `Libro` ha `titolo` — ma tutti espongono `getCosto()` e `getPrezzo()` come richiesto dal contratto.

Il Consumer non sa quale di questi tre oggetti sta usando — vede solo `Prodotto` .

4. La classe `FactoryMethod`



Java

```
1 package designPattern.factoryMethod;
2 public class FactoryProdotto {
3
4     // qui imposto la logica creazionale
5     @SuppressWarnings("deprecation")
6     public static Prodotto getProdotto(String tipoProdotto) {
7
8         if(tipoProdotto.equals("Libro")) {
9             return new Libro("titolo del libro", 10, 15);
10        }else {
11            return new TShirt(10,25, "M");
12        }
13    }
```

È il cuore del pattern.

Il metodo `getProdotto()` è statico — **non si istanzia mai la Factory, la si usa direttamente**.

Riceve in input una stringa con il nome del tipo di prodotto da creare e restituisce un oggetto di tipo `Prodotto` — il tipo astratto, non quello concreto.

Il Consumer non vede mai `new Zaino()` o `new TShirt()` — quella logica è incapsulata qui dentro.

Da notare come in questo caso si usi il modo "classico" di implementare la factory un `if/else` che in base alla stringa in input crea l'oggetto corretto. Funziona, ma è rigido:

- **ogni volta che aggiungi un nuovo `ConcreteProduct` devi modificare la `Factory`.**

5. La classe `Consumer` — il consumatore



Java

```
1 package designPattern.factoryMethod;
2
3 public class Consumer {
4
5     public static void main(String[] args) {
6         // chiedo alla factory di produrre un prodotto concreto, senza
        // chiamare il costruttore
7     }
```



```

7      Prodotto product =
      FactoryProdotto.getProdotto("designPattern.factory.Zaino");
8      System.out.println(product);
9      System.out.println(product.getPrezzo());
10     }
11 }

```

Il Consumer chiede alla Factory di creare un prodotto passando il nome della classe come stringa.

Riceve un oggetto di tipo `Prodotto` e lo usa tramite l'interfaccia — chiama `getPrezzo()` senza sapere se sta parlando con uno `Zaino`, una `TShirt` o un `Libro`.

Nota il disaccoppiamento totale:

il Consumer non importa nessuna delle classi concrete — non vede `Zaino`, non vede `TShirt`, non vede `Libro`. Vede solo `Prodotto` e `FactoryProdotto`.

Se domani aggiungiamo un nuovo prodotto `Scarpa`, il Consumer non va toccato.

Java Reflection

Abbiamo visto che il modo "classico" di implementare la Factory con un `if/else` è rigido:

- **ogni volta che aggiungi un nuovo `ConcreteProduct` devi modificare la Factory.**

La **Java Reflection** risolve questo problema permettendo di istanziare una classe **senza conoscerne il nome a compile time**.

Il concetto è semplice:

- **normalmente per istanziare una classe scriviamo `new Zaino()` — ma questo significa che il nome della classe deve essere noto a compile time.**
- **Con la Reflection invece possiamo fare la "new" di una classe il cui nome lo scopriamo solo a runtime, ad esempio leggendolo da una stringa.**

1. Caricare la classe per nome — `Class.forName()`



Java

```
1 Class cl = Class.forName("designPattern.factoryMethod.Zaino");
```

- Questo metodo importa dinamicamente la classe il cui nome è contenuto nella stringa
- restituisce un oggetto di tipo `Class` che rappresenta la classe caricata.

2. Istanziare l'oggetto — `newInstance()`

Ora che abbiamo l'oggetto `cl:Class`



Java

```
1 Object obj = cl.newInstance();
```

Questo metodo istanzia la classe appena caricata invocando il **costruttore a zero argomenti**:

- per questo motivo tutte le classi che vogliamo istanziare tramite **Reflection** devono avere un costruttore vuoto.
- `newInstance()` restituisce un `Object` generico, quindi da solo permette di usare solo i metodi di `Object`.

Il problema del tipo generico — il cast all'interfaccia:

- Se `newInstance()` restituisce un `Object`, come facciamo a chiamare `getPrezzo()` e `getCosto()`? La soluzione è il **cast all'interfaccia** `Prodotto`:



Java

```
1 String nomeClasse = "designPattern.factoryMethod.Zaino"; // nome
  della classe da istanziare
2
3 Class cl = Class.forName(nomeClasse); // carico la classe per
  nome
4 Prodotto p = (Prodotto) cl.newInstance(); // istanzio e casto
  all'interfaccia
5
6 p.getPrezzo(); // ora posso usare i metodi dell'interfaccia
7 p.getCosto();
```

Sappiamo che la classe passata implementa `Prodotto` — quindi il cast è sicuro.

A questo punto possiamo chiamare tutti i metodi dell'interfaccia sull'oggetto, indipendentemente da quale classe concreta è stata istanziata.

Il risultato nella Factory Applicando la Reflection alla `FactoryProdotto`, il metodo `getProdotto()` diventa completamente generico:



Java

```
1  public static Prodotto getProdotto(String tipoProdotto) {
2      try {
3          Class classe = Class.forName(tipoProdotto);
4          Prodotto ob = (Prodotto) classe.newInstance();
5          return ob;
6      } catch (ClassNotFoundException e) {
7          e.printStackTrace();
8      } catch (InstantiationException | IllegalAccessException e) {
9          e.printStackTrace();
10     }
11     return null;
12 }
```

Non c'è più nessun `if/else` — la Factory non conosce `Zaino`, `TShirt` o `Libro`.

1. Riceve una stringa,
2. carica la classe corrispondente,
3. la istanzia e la restituisce come `Prodotto`.

Se domani aggiungiamo `Scarpa` al sistema, la Factory non va toccata — basta passare la stringa `"designPattern.factoryMethod.Scarpa"` e la Reflection pensa al resto.

Nota il collegamento con Spring:

quando Spring legge l'annotation `'@Component'` su una classe, fa esattamente la stessa cosa —

carica la classe per nome tramite Reflection, la istanzia tramite il costruttore a zero argomenti (ecco perché è obbligatorio!) e la registra nel Container.

La Java Reflection è il meccanismo che sta sotto a tutta la magia dell'IoC.

2. Singleton

Il problema che risolve:

- A volte è necessario garantire che esista una ed una sola istanza di una certa classe in tutto il sistema.
- Pensiamo alla classe `Database` che abbiamo scritto nell'esercizio JDBC:
 - non ha senso creare più istanze di quella classe — ogni istanza aprirebbe una nuova connessione al database, spreca risorse e rischiando comportamenti inconsistenti.
 - Lo stesso vale per i `Logger`, i `Config Server` e in generale per tutti i componenti che gestiscono risorse condivise.

La soluzione:

- L'idea è impedire che chiunque possa creare liberamente oggetti di quella classe tramite `new`.

In Java questo si implementa con tre elementi che lavorano insieme:

1. Costruttore privato:

- nessuno può chiamare `new MiaClasse()` dall'esterno.
- L'unico modo per ottenere un'istanza è attraverso il metodo dedicato.

2. Attributo privato e statico del tipo della classe:

- è qui che viene conservata l'unica istanza.
- Essendo `static`, appartiene alla classe e non a nessun oggetto specifico — esiste una sola volta in memoria per tutta la durata del programma.

3. Metodo pubblico e statico che restituisce l'istanza:

- è l'unico punto di accesso all'istanza.
- Se l'istanza non esiste ancora la crea, altrimenti restituisce quella già esistente.



Java

```
1 public class Database {  
2     // 2. attributo privato e statico - conserva l'unica istanza  
3     private static Database istanza = null;  
4 }
```

```

5      // 1. costruttore privato – nessuno può fare new Database()
      dall'esterno
6      private Database() { }
7
8      // 3. metodo pubblico e statico – unico punto di accesso
      all'istanza
9      public static Database getInstance() {
10         if(istanza == null) {
11             istanza = new Database(); // crea l'istanza solo la
      prima volta
12         }
13         return istanza; // restituisce sempre la stessa istanza
14     }
15 }

```

Il meccanismo è semplice:

- la prima volta che qualcuno chiama `Database.getInstance()`, `istanza` è `null` — quindi viene creata.
- Tutte le volte successive, `istanza` non è più `null` — quindi viene restituita quella già esistente.

In questo modo è **fisicamente impossibile** avere più di un'istanza di `Database` in tutto il sistema.

Nota il collegamento con Spring:

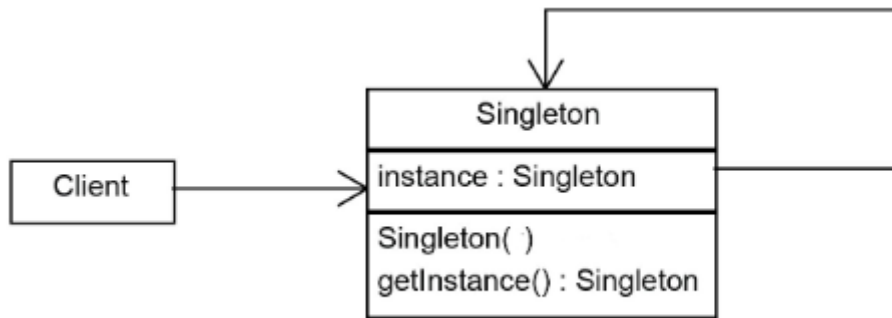
i bean annotati con `@Component`, `@Service` e `@Repository` sono **Singleton per default** — **Spring crea una sola istanza di ogni bean e la riusa per tutte le richieste.**

È esattamente il Singleton pattern applicato su scala architetturale.

Diagramma Singleton

Il diagramma mostra una struttura molto semplice — il Singleton è uno dei pattern più compatti da implementare, ma uno dei più potenti in termini di impatto architetturale.

Per capire questo pattern analizziamo l'immagine del diagramma:



1. Abbiamo una classe `Client` che partecipa a un link di associazione con la responsabilità verso la classe `Singleton`

☞ Il Client inteso come colui che ha l'oggetto della classe o interfaccia di cui è responsabile

Nei casi dei Design Pattern, quando si parla di **Client**, non ci si riferisce a un Client inteso nella accezione canonica del termine (es: il Client nell'architettura **Client - Server**), ma alla classe (può essere anche la classe con il `Main` di Java) che implementa l'oggetto della classe o interfaccia da cui parte il Design Pattern

1. La classe `Singleton`:

- ha un attributo `instance` tipizzato a se stesso
- un metodo `Singleton()` senza parametri e senza ritorno
- E un metodo `getInstance()` che ritorna un oggetto della classe `Singleton`
- Difatti questa classe ha un link di associazione che richiama se stessa
Cosa significa tutto ciò?

Il Client non può creare oggetti della classe direttamente — il costruttore è privato.

L'unico modo per ottenere l'istanza è chiamare il metodo statico `getInstance()`.

Il metodo `getInstance()` può avere due comportamenti:

1. **Lazy initialization:**

- **l'istanza viene creata solo alla prima richiesta.**
- **È il comportamento che abbiamo visto nell'esempio precedente con il controllo `if(istanza == null)`.**

- Il vantaggio è che se nessuno chiede mai l'istanza, non viene mai creata — risparmio di risorse.
- È l'approccio più comune.



Java

```
1 public static Database getInstance() {
2     if(istanza == null) {
3         istanza = new Database(); // creata solo alla prima
        chiamata
4     }
5     return istanza;
6 }
```

2. Eager initialization:

- l'istanza viene creata al caricamento della classe, prima ancora che qualcuno la richieda.
- È più semplice da implementare e non ha problemi di thread safety, ma crea l'oggetto anche se non verrà mai usato.



Java

```
1 private static Database istanza = new Database(); // creata
    subito al caricamento
2
3 public static Database getInstance() {
4     return istanza; // restituisce sempre quella già creata
5 }
```

👁 La scelta tra i due approcci dipende dal contesto:

se l'oggetto è pesante da creare e potrebbe non essere necessario, si preferisce la **lazy initialization**.

Se invece l'oggetto è leggero e sarà sicuramente usato, la **eager initialization** è più semplice e sicura.

Pattern Strutturali

3. Composite ⚠ Da portare all'esame

Il Composite è un pattern molto particolare che risolve una categoria specifica di problemi — **quelli in cui gli oggetti hanno una struttura ricorsiva ad albero.**

Ricordi gli algoritmi ricorsivi che abbiamo visto a inizio corso?

La ricorsione è la soluzione più elegante per certi problemi — quando un problema può essere scomposto in sotto-problemi della stessa natura.

Il Composite è esattamente questo, ma applicato a livello di progettazione degli oggetti: è la ricorsione a livello di design.

Il problema che risolve:

Immagina di dover modellare la struttura di un **filesystem**:

- **hai cartelle che contengono file, ma anche cartelle che contengono altre cartelle, che a loro volta contengono altri file e altre cartelle.**
- **È una struttura ad albero naturalmente ricorsiva.**

Il problema è duplice:

- **Ridondanza nel codice** — **senza questo pattern dovresti scrivere codice diverso per gestire le foglie (i file) e i nodi composti (le cartelle), creando duplicazione e complessità inutile**
- **Accesso non uniforme** — **vorresti poter chiamare `calcola()` o `stampa()` su qualsiasi elemento dell'albero — sia su una singola foglia che su un intero sottoalbero — senza dover sapere a priori su quale dei due stai agendo**

La soluzione:

Si realizza un'interfaccia generica `Componente` che modella sia gli elementi singoli (le foglie) che la composizione (i nodi).

L'interfaccia viene implementata da due classi:

- `Foglia (Leaf)` — **rappresenta gli elementi terminali dell'albero, quelli che non contengono altri elementi (es. un file).**
- `Composizione (Composite)` — **rappresenta i nodi che possono contenere altri nodi.**
- `Componente (Componente)` — **sia foglie che altre composizioni (es. una cartella).**

Il punto chiave è:

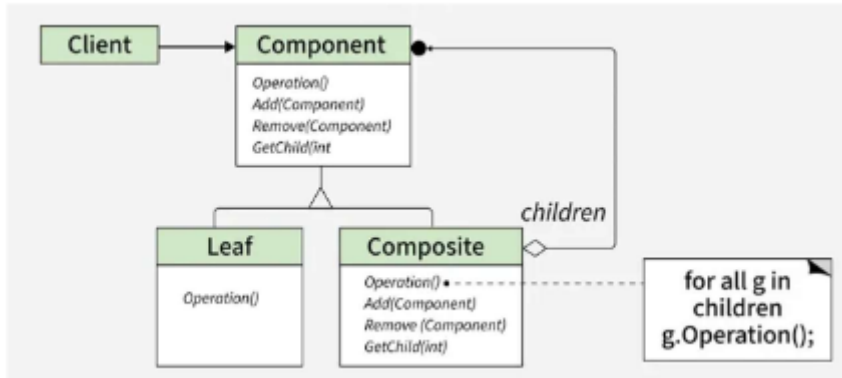
- **la `Composizione` contiene una lista di `Componente` — non di `Foglia` o di `Composizione` specificamente, ma dell'interfaccia generica.**

Questo è ciò che rende il pattern ricorsivo:

- una `Composizione` può contenere altre `Composizione`, che a loro volta ne contengono altre, formando un albero di profondità arbitraria.

Diagramma del Composite

Per capire meglio questo design pattern analizziamo l'immagine



I ruoli dei tre elementi sono:

1. Classe `Client` :

- Partecipa a un link di associazione con responsabilità verso l'interfaccia / classe astratta `Component`.
- La classe client non sa nulla della struttura del design pattern, implementa l'oggetto della classe `Component`.

2. Classe astratta/interfaccia `Component`

- è l'interfaccia generica, di base una classe astratta, che definisce il contratto comune per tutti gli elementi dell'albero.
- Dichiarare il metodo di business `Operation()` che verrà implementato sia dalle foglie che dalle composizioni.
- Gli altri metodi presenti nell'interfaccia — come `add()`, `remove()`, `getChild()` — non sono operativi ma servono a gestire i sottocomponenti e hanno senso solo per `Composite`, non per `Leaf`.

Ricorda:

`Component` 9 volte su 10 è una classe astratta, non un interfaccia

3. Classe `Composite`

- è il nodo dell'albero che può contenere altri `Component`.
- Implementa `Operation()` delegando la chiamata a tutti i suoi sottocomponenti, che a loro volta la delegano ai propri, fino ad arrivare alle foglie.

- È una chiamata ricorsiva che scende lungo l'albero fino agli elementi terminali.

4. Classe `Leaf`:

- è l'elemento terminale dell'albero, quello che non può contenere altri elementi.
- Implementa `Operation()` con la sua logica specifica — è qui che avviene il lavoro concreto.

Il diagramma del Composite mostra due relazioni fondamentali che lavorano insieme:

1. Relazione is-a:

- sia `Leaf` che `Composite` implementano l'interfaccia `Component`.
- Entrambi sono un `Component`:
 - questo è ciò che permette al Client di trattarli in modo uniforme senza sapere con quale dei due sta interagendo.

2. Relazione has-a:

- `Composite` aggrega una collezione di `Component`.
- Non aggrega specificamente `Leaf` o altri `Composite`, ma il tipo astratto `Component` — ed è esattamente questa la chiave della ricorsione.
 - Una `Composite` può contenere foglie, altre composizioni, o entrambe, senza limiti di profondità.

Guardando il diagramma non si può sapere quanto è profonda la ricorsione — dipende interamente da come viene costruito l'albero a runtime.

≡ Analogia con il filesystem:

`Component` è il concetto generico di "elemento del filesystem", `Leaf` è un file, `Composite` è una cartella.

Una cartella può contenere file e altre cartelle — esattamente come `Composite` può contenere `Leaf` e altri `Composite`. Quando chiedi la dimensione totale di una cartella, il sistema delega il calcolo a tutti gli elementi contenuti, che a loro volta delegano ai propri, fino ad arrivare ai singoli file che restituiscono la loro dimensione.

È il Composite pattern in azione.

Esempio Pratico — Composite in Java: Gestione di un Teatro

L'esercizio modella la struttura di un teatro — un esempio perfetto di struttura ad albero ricorsiva:

- un teatro ha settori, i settori possono contenere altri settori, e alla fine della gerarchia ci sono le zone con i loro posti.

Vediamo come le classi si mappano sui ruoli del pattern:

SettoreComponent — il Component



Java

```
1 public abstract class SettoreComponent {
2     private int id;
3
4     public void add(SettoreComponent sc) throws LeafException {};
5     public boolean removeSettore(SettoreComponent sc) throws
    LeafException { return false; };
6 }
```

- È la classe astratta che definisce il contratto comune per tutti gli elementi della gerarchia.
- Dichiarare i metodi `add()` e `removeSettore()` con implementazioni vuote di default — non astratti, perché la `Leaf` non dovrebbe implementarli ma solo lanciarli come eccezione.
- Contiene l' `id` che identifica ogni settore, sia esso un nodo composito o una foglia.

SettoreComposite — il Composite



Java

```
1 public class SettoreComposite extends SettoreComponent {
2     private List<SettoreComponent> contiene = new ArrayList<>();
3
4     @Override
5     public void add(SettoreComponent sc) throws LeafException {
6         this.getContiene().add(sc);
7     }
8
9     @Override
10    public boolean removeSettore(SettoreComponent sc) throws
    LeafException {
11        return this.getContiene().remove(sc);
12    }
```

```
12     }
13 }
```

- È il nodo dell'albero — **aggrega una** `List<SettoreComponent>` , **non una lista di** `SettoreComposite` **o di** `Zona` **specificamente, ma del tipo astratto** `SettoreComponent` .
- Questo è il cuore della ricorsione:
 - **un** `SettoreComposite` **può contenere altri** `SettoreComposite` **o** `Zona` , **senza limiti di profondità.**
 - **I metodi** `add()` **e** `removeSettore()` **aggiungono e rimuovono elementi dalla lista.**
`Zona` — la `Leaf`



Java

```
1  public class Zona extends SettoreComponent {
2      private List<Posto> posti = new ArrayList<>();
3
4      @Override
5      public void add(SettoreComponent sc) throws LeafException {
6          throw new LeafException("Errore! La zona non può
aggiungere zone o posti");
7      }
8
9      @Override
10     public boolean removeSettore(SettoreComponent sc) throws
LeafException {
11         throw new LeafException("Errore! La zona non può
rimuovere zone o posti");
12     }
13
14     public void addPosto(Posto p) { ... }
15     public boolean removePosto(Posto p) { ... }
16 }
```

- È la foglia della gerarchia del pattern — **non può contenere altri** `SettoreComponent` .
- **Quando qualcuno tenta di chiamare** `add()` **o** `removeSettore()` **su una** `Zona` , **viene lanciata una** `LeafException` — **il meccanismo che impedisce alla foglia di comportarsi come un nodo composito.**

- Zona ha però una propria lista di Posto gestita con metodi dedicati `addPosto()` e `removePosto()` — ma questi non fanno parte della gerarchia del Composite, sono una gestione interna della Zona.

Posto — oggetto a sé stante



Java

```
1      public class Posto {
2          private int numeroPosto;
3          private char fila;
4          public Posto(int numeroPosto, char fila) {
5              this.setNumeroPosto(numeroPosto);
6              this.setFila(fila);
7          }
8          public Posto() {}
9          public int getNumeroPosto() {
10             return numeroPosto;
11         }
12         public void setNumeroPosto(int numeroPosto) {
13             this.numeroPosto = numeroPosto;
14         }
15         public char getFila() {
16             return fila;
17         }
18
19         public void setFila(char fila) {
20             this.fila = fila;
21         }
22         @Override
23         public String toString() {
24             return "Posto :{ numero posto : " + getNumeroPosto() + ",
25             fila posto : " + getFila() + "}";
26         }
27     }
```

- Posto non fa parte della gerarchia del Composite — non estende `SettoreComponent`.
- È un oggetto autonomo che rappresenta il dettaglio più granulare del sistema — un singolo posto in una zona.
- La sua gestione è completamente delegata alla Zona tramite i suoi metodi `addPosto()` e `removePosto()`.

LeafException — la guardia della Leaf



Java

```
1 public class LeafException extends Exception {  
2     public LeafException(String msg) { super(msg); }  
3 }
```

È una **checked exception** custom che viene lanciata ogni volta che si tenta di usare i metodi strutturali del Composite su una Zona .

Essendo checked, **il compilatore obbliga chi chiama** `add()` **o** `removeSettore()` **a gestirla con un try-catch — rendendo esplicito nel codice il fatto che queste operazioni possono fallire se invocate su una foglia.**

test — il Client



Java

```
1 public class test {  
2     public static void main(String[] args) throws LeafException {  
3         // ...  
4     }  
5 }
```

Il `main` costruisce la struttura ad albero del teatro passo per passo. Vale la pena seguire il flusso per capire come il pattern prende vita nel codice.

Prima parte — costruzione del primo settore:



Java

```
1 Posto p = new Posto(1, 'C');  
2 Posto p2 = new Posto(2, 'A');  
3 Posto p3 = new Posto(3, 'D');  
4  
5 Zona z = new Zona(2, new ArrayList<Posto>());  
6 z.addPosto(p);  
7 z.addPosto(p2);  
8 z.addPosto(p3);
```

Si creano tre posti e si aggiungono alla Zona z tramite `addPosto()` — il metodo interno della Zona , non quello del pattern.



Java

```
1 List<SettoreComponent> listaSC = new ArrayList<SettoreComponent>();
2 SettoreComponent sc = new SettoreComposite(1, listaSC);
3 sc.add(z);
```

Si crea un `SettoreComposite` e lo si riferenzia con una reference di tipo `SettoreComponent` — il tipo astratto. Questo è il disaccoppiamento in azione:

- `sc` **non sa di essere un** `SettoreComposite`, **vede solo l'interfaccia** `SettoreComponent`.

Si aggiunge poi `z` al settore tramite `sc.add(z)` — il metodo del pattern.



Java

```
1 System.out.println(sc.toString());
```

Stampa la struttura del settore con la zona e i suoi posti.

Seconda parte — aggiunta di un secondo settore:



Java

```
1 Posto p4 = new Posto(4, 'F');
2 Posto p5 = new Posto(6, 'T');
3 Posto p6 = new Posto(8, 'Z');
4
5 Zona z2 = new Zona(3, new ArrayList<Posto>());
6 z2.addPosto(p4);
7 z2.addPosto(p5);
8 z2.addPosto(p6);
9 sc.add(z2);
10 System.out.println(sc.toString());
```

Si crea una seconda zona `z2` e si aggiunge al settore.

👁 Nota importante:

il Client dichiara `sc` **come** `SettoreComponent` **— non come** `SettoreComposite`.

Questo significa che quando chiama `sc.add(z)` **non sa se sta** **aggiungendo a un** `SettoreComposite` **o a una** `Zona`.

Se `sc` fosse una `Zona`, riceverebbe una `LeafException`.

È il polimorfismo del Composite in azione — il Client opera sull'interfaccia senza sapere cosa c'è dietro.

La Struttura ad Albero del Teatro

Mettendo tutto insieme, la struttura che si può costruire con questo pattern è:



CSS

```
1  Teatro (SettoreComposite)
2  |— Settore A (SettoreComposite)
3  |   |— Zona A1 (Zona/Leaf)
4  |   |   |— Posto {1, 'A'}
5  |   |   |— Posto {2, 'A'}
6  |   |— Zona A2 (Zona/Leaf)
7  |   |   |— Posto {1, 'B'}
8  |— Settore B (SettoreComposite)
9  |   |— Zona B1 (Zona/Leaf)
10 |   |   |— Posto {1, 'C'}
```

Il Client può navigare questa struttura chiamando i metodi di

`SettoreComponent` **senza sapere se sta interagendo con un**

`SettoreComposite` **o con una** `Zona` **— è il polimorfismo applicato alla**
ricorsione.

✓ Conseguenze del Pattern Composite

Accesso uniforme:

- **foglie e nodi compositi vengono trattati in modo uniforme tramite l'interfaccia `Component`.**
- **Il Client non deve sapere con quale dei due sta interagendo — chiama sempre gli stessi metodi e il polimorfismo pensa al resto.**

Nessuna ripetizione nel diagramma:

- **la struttura ad albero non genera ridondanza nel codice.**
- **Non esiste un metodo diverso per gestire le foglie e uno per gestire i nodi — esiste un'unica interfaccia che vale per entrambi.**
- **È la soluzione DRY (Don't Repeat Yourself) applicata alla progettazione.**

Profondità dinamica:

- la profondità della struttura non è fissa a compile time ma si decide direttamente al momento della creazione degli oggetti a runtime.
- Nell'esercizio del teatro, il Client costruisce l'albero aggiungendo settori e zone liberamente — il pattern non impone nessun limite sulla profondità o sul numero di nodi.

In sintesi:

il Composite risolve elegantemente il problema della ricorsione a livello di design:

- una sola interfaccia, nessuna ripetizione, profondità arbitraria.

È uno dei pattern più potenti e allo stesso tempo più eleganti del catalogo GoF.

Pattern Comportamentali

4. Observer

Il problema che risolve:

A volte si ha un oggetto il cui cambiamento interno deve produrre delle azioni — e si vuole che queste azioni siano configurabili dinamicamente, senza dover modificare l'oggetto stesso ogni volta.

È il contesto classico della programmazione ad eventi: quando succede qualcosa, qualcun altro deve reagire.

Event Listener in JS

Lo abbiamo già incontrato senza saperlo — quando in JavaScript associavamo un **event listener** a un bottone HTML:

JS JavaScript

```
1 button.addEventListener('click', () => { ... });
```

Il bottone è l'osservato, l'event listener è l'osservatore. Quando il bottone viene cliccato — cambiamento interno — l'event listener reagisce eseguendo il suo comportamento.

La soluzione: Si creano due classi con ruoli distinti:

1. **L'Osservato** — è l'oggetto che subisce i cambiamenti.

- Ha la responsabilità di:
 - Registrare gli osservatori interessati ai suoi cambiamenti
 - Avvisarli quando si verifica un evento

2. **L'Osservatore:**

- è l'oggetto che reagisce ai cambiamenti dell'Osservato.
- Esegue comportamenti specifici in funzione dell'evento che si è verificato.

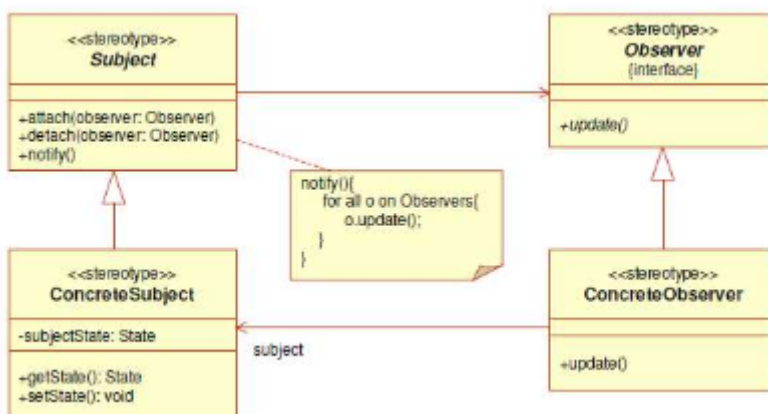
Il vantaggio fondamentale di questa soluzione è che il **numero e il tipo degli osservatori non deve essere noto a priori**:

- si possono aggiungere o rimuovere osservatori dinamicamente a runtime, senza toccare il codice dell'osservato.

L'osservato sa solo che esistono degli osservatori e che deve avvisarli — non sa quanti sono né cosa fanno.

Diagramma dell'Observer

Guardando il diagramma, il pattern si articola su quattro elementi:



1. **Subject** — l'Osservato astratto:

- È la classe astratta che definisce il contratto per tutti gli osservati.
- Dichiarare tre metodi fondamentali:

- `attach(observer: Observer)` — registra un nuovo osservatore
- `detach(observer: Observer)` — rimuove un osservatore
- `notify()` — avvisa tutti gli osservatori registrati

2. `ConcreteSubject` — l'Osservato concreto:

- **Estende** `Subject` e contiene lo stato che può cambiare — `subjectState: State`.
- **Espone** `getState()` e `setState()` per leggere e modificare lo stato.
- **Quando lo stato cambia, chiama** `notify()` che scatena la reazione di tutti gli osservatori registrati.

3. `Observer` — l'Osservatore astratto:

- **Partecipa a un link di associazione con** `Subject`, la quale ha la responsabilità verso questa classe:
 - **Difatti** `Subject` nel suo metodo `notify()` invoca il metodo `update()` di `Observer` tramite oggetto `o:Observer`
- **È un'interfaccia che dichiara un solo metodo:** `update()`.
- **È il contratto che tutti gli osservatori concreti devono rispettare.**

`ConcreteObserver` — l'Osservatore concreto:

- **Implementa** `Observer` e specializza il metodo `update()` con il comportamento specifico da eseguire quando viene notificato.
- **Partecipa a un link di associazione con responsabilità verso** `ConcreteSubject`:
 - **per poterne leggere lo stato aggiornato tramite** `getState()`.

 Il cuore del pattern è il metodo `notify()`, che internamente fa:



Java

```
1  >void notify() {
2  for(Observer o : observers) {
3      o.update(); // avvisa ogni osservatore registrato
4  }
5  }
```

Quando il `ConcreteSubject` **cambia stato, chiama** `notify()` **che itera su tutti gli osservatori registrati e chiama** `update()` **su ognuno.**

Ogni `ConcreteObserver` reagisce al cambiamento eseguendo il suo

comportamento specifico — **che può includere leggere il nuovo stato dell'osservato tramite `getState()` e modificarsi di conseguenza.**

🔗 Il punto chiave è che il `Subject` **non sa nulla dei** `ConcreteObserver` — **conosce solo l'interfaccia** `Observer`.

Questo significa che si possono aggiungere nuovi tipi di osservatori senza mai toccare il codice del `Subject` — esattamente come in JavaScript si possono aggiungere più event listener allo stesso bottone senza modificare il bottone stesso.

✓ **Conseguenze del Pattern Observer**

L'Observer è esterno e indipendente:

- **l'osservatore non è parte integrante del `Subject` ma un componente esterno che può essere configurato, aggiunto o rimosso dinamicamente senza toccare il codice dell'osservato.**
- È esattamente il principio di **Single Responsibility**:
 - **il `Subject` si occupa di gestire il proprio stato, gli `Observer` si occupano di reagire ai cambiamenti.**

Un Observer può osservare più Subject:

- **uno stesso osservatore può essere registrato su più oggetti osservati contemporaneamente.**
- Ad esempio un logger che registra i cambiamenti di stato potrebbe essere registrato su tutti i microservizi del sistema.

L'Observer è autorizzato ad agire sul Subject:

- **l'osservatore non si limita a leggere lo stato dell'osservato tramite `getState()` — se necessario può anche modificarlo tramite `setState()`.**
- **Il `Subject` autorizza implicitamente questa possibilità nel momento in cui registra l'osservatore.**

Observer vs Proxy — Una Distinzione Importante

L'Observer può sembrare simile al **Proxy** — entrambi coinvolgono un oggetto che "si mette in mezzo" rispetto a un altro.

La differenza fondamentale però è:

- Nel **Proxy**:
 - **il soggetto filtrato non sa nulla del proxy che lo avvolge.**
 - **Il proxy intercetta le chiamate all'insaputa dell'oggetto originale.**
- Nell'**Observer** — l'osservato sa dell'osservatore:
 - **è lui stesso a registrarlo tramite `attach()` e ad avvisarlo tramite `notify()`.**
 - **Non c'è nulla di nascosto — è una relazione esplicita e bidirezionale.**

 **In sintesi: il Proxy è trasparente all'oggetto originale, l'Observer è una relazione consapevole e dichiarata tra osservato e osservatore.**

Proxy